Infrastructure-centric, NetworkServer-agnostic LoRaWAN Roaming

Giovanni Merlino*†, Rafael Asorey-Cacheda‡, Luca D'Agati*, Francesco Longo*†,
Antonio-Javier Garcia-Sanchez‡, Joan Garcia-Haro‡ and Antonio Puliafito*†
* Department of Engineering, University of Messina, Italy
Email: {gmerlino,flongo,apuliafito}@unime.it
luca.dagati@studenti.unime.it
† CINI: National Interuniversity Consortium for Informatics, Rome, Italy
‡ Department of Information and Communication Technologies, Universidad Politécnica de Cartagena, Spain
Email: {rafael.asorey,antoniojavier.garcia,joang.haro}@upct.es

***Abstract*—The development of Low-Power Wide-Area Networks (LPWAN) has significantly boosted the uptake of Internet of Things technologies, fostering their adoption in such domains as Smart Cities and Industry 4.0. In the context of LPWANs, LoRa is one of the most promising developments, experiencing massive growth in recent years. This situation has led to the steadily growing trend of redundant deployments across locations, or scenarios, where it would be more natural to be able to leverage (e.g., rent) LoRa infrastructure belonging to a third party. Key to reversing this unsustainable trend is roaming approaches. Although the LoRa specifications define mechanisms for roaming, gaps and challenges make roaming difficult to deploy and operate in real-world scenarios. This paper analyzes these concerns and proposes a novel architecture compliant with the LoRa specification to design and accommodate roaming services in a consistent, efficient, and scalable way.**

***Index Terms*—LoRaWAN, Roaming, Service Level Agreement, Smart Contract.**

I. Introduction

In the context of smart cities and smart agriculture, to name just a few domains where the LoRaWAN ecosystem is relevant and potentially disruptive, the missing link appears to be a relatively late interest and effort in supporting roaming seamlessly and efficiently, especially for cash-strapped communities, where deployments may be carried out in subsequent waves. However, service needs to be available from the earliest stages. Moreover, mobility is essential for LoRaWAN to compete with 5G while filling a niche of its own, e.g., grassroots services and operations, without cooperation from incumbent telcos.

There are currently two LoRaWAN specifications, 1.0 and 1.1. The main difference between these versions is in the additional roaming support and security features of version 1.1. However, the 1.1 specification has several design issues blocking the LoRa Alliance from timely certifying devices that implement this version [1]. Thus, LoRaWAN 1.1 is not yet ready for mass production and deployment, and it will probably take years for any significant adoption. In addition, other aspects limit the coexistence of the two versions. Thus, version 1.1 implements backward compatibility with version 1.0. However, this support is limited [2]. In practice, within the same LoRaWAN network, the two versions cannot coexist. This implies that if a single device does not support version 1.1, all the other devices will have to use version 1.0. So this simple fact, in a market with millions of devices that only support the 1.0 specification and the absence of a certification program, is one of the most significant barriers to adopt the 1.1 specifications.

In summary, in the medium term, LoRaWAN network deployment will only be available exclusively with support for the 1.0 spec. That is why a pragmatic approach should be followed, and any incremental (and, where possible, backward-compatible) proposal around this technology has to be based on the functionalities provided by version 1.0 of the protocol. The proposal stems from this line of reasoning, so all limitations built into the 1.0 specification are factored in. However, these constraints should not hinder supporting roaming and, thus, the mobility of end devices.

In particular, this work addresses the issues with the current limitations in the LoRaWAN standards and markets by taking a three-fold approach to LoRaWAN-supported mobility: i) decouple the deployment and operations of Gateways (GWs) from those of (LoRaWAN) networks; ii) establish Gateway-to-Network connections on-demand, according to actual traffic (in mobility); iii) support agreements dynamically, by automatically settling provisions and fees through Smart Contracts (SCs).

This paper proposes a novel roaming approach, enabling the forwarding of packets to a Network Server (NS) via an unidentified GW without involving predefined agreements among NSs. Moreover, a design to authorize a forwarding GW through service level agreements (SLA) is envisioned here. Albeit out of scope for this work, SCs in particular are being considered for automatic SLA negotiation. The design is outlined in this work by describing the protocol. Related works about roaming and SCs are presented in Section II. An overview of typical (i.e., standardized) LoRaWAN roaming facilities is provided in Section III, whereas our roaming proposal is described in Section IV. A performance evaluation

TABLE I
ACRONYMS.

List of acronyms used in this paper	
Abbreviation	Definition
ACL	Access Control List
AS	Application Server
BC/SC	Blockchain/smart contract
DevAddr	Device Address
DNS	Domain Name System
ED	End Device
fNS	Forwarding Network Server
fSC	Forwarding Smart Contract
fSvcProvider	Forwarding Service Provider
GW	Gateway
IoT	Internet of Things
JS	Join Server
LPWAN	Low Power Wide Area Networks
MIC	Message Integrity Code
NwkID	Network ID
NS	Network Server
SC	Smart Contract
SLA	Service Level Agreement
sNS	Serving Network Server
sSC	Serving Smart Contract
sSvcProvider	Serving Service Provider

of the proposed architecture and its implementation is reported in Section V. Section VI closes the work and provides hints about future work. To improve the document's readability, the acronyms used are listed in Table I.

II. Related Work

Roaming solutions in LoRaWAN have been subjects of research in literature. In [3], the authors proposed a roaming LoRaWAN architecture that employs time-tested Internet infrastructures such as the Domain Name System (DNS) and Public Key Infrastructure (PKI) to enable interconnection and LoRa End-Devices (EDs) security. The proposed architecture included a PKI-based authentication and authorization (AA) framework that enables secure onboarding of the IoT ED and allows roaming between different LoRaWAN networks without needing any prior interconnection agreement. In [4], the authors proposed a modification to IoTRoam by introducing a new entity called DNS Broker responsible for the decentralized orchestration of roaming between different operators using private and secure DNS resolutions. This approach is based on introducing new entities called Brokers that control the relationship between the network operator and device owner, eliminating the need for the latter to handle complex identifiers and reduce the number of interactions to determine a serving NS. In [5], the authors' proposal allowed interoperability between the 5G network and LoRaWAN so that multi-RA Internet of Things (IoT) devices (radio access technologies) can exploit both technologies interchangeably, thus fostering new IoT mobility and roaming use cases for LPWAN employing SIM-based authentication in the 5G network via the LoRaWAN protocol. In [6], the authors introduced an extension of the SCHC (Static Context Header Compression) mechanism, called SCHC mobile (MSCHC), capable of handling mobility based on compressed lightweight mobile IPv6 messages, guaranteeing session continuity when the ED is roaming between different operators during data transmission and reception. In [7], the authors offered a solution to facilitate roaming since you only need to join the new NS without connecting the Application Server (AS) again. In [8], authors handled the challenge of roaming across several cooperative shipping companies and their LoRaWAN networks with the aid of Artificial Intelligence (AI) and Machine Learning (ML) techniques; the roaming proposal is still in the testing phase. In [9], the authors presented a new LoRaWAN roaming project to enable inter-operator roaming based on DNS resolution and end-device context migration between networks. They designed several roaming mechanisms, namely DNS resolution and the introduction of add-ons, such as a Master Agent, which creates an interconnection between LoRaWAN networks, and a Local Agent, which connects to NS, and the migration of the context between home networks and those visited. In [10], the authors focused on enhancing LoRaWAN connectivity for roaming devices by integrating a machine learning predictor and DNS prefetch to gather the information needed to establish the connection before the device enters coverage, thus reducing the overall latency for establishing the connection.
None of these solutions address the enabling of LoRaWAN roaming at the Gateway level, as we propose in this paper.

Moreover, much effort has been put into exploiting Blockchain/Smart Contract (BC/SC) technologies in the context of LoRaWAN [11], [12]. As an example, in [13], an Over-The-Air (OTA) procedure has been designed for LoRaWAN EDs based on the use of the Interplanetary File System (IPFS).

More specifically, a few works in the literature investigated the use of BC/SC technologies to solve security challenges related to mobility and roaming in LoRaWAN [14]. In [15], [16], authors conceived a BC/SC-backed framework for the LoRaWAN join procedure that prevents jamming and reply attack threats, which the LoRaWAN standard protocol is vulnerable to. The approach also guarantees availability and immutability as side-effects of the decentralization of ED authentication information.

Such information is saved in the blockchain network thus, failure of some NSs will not prevent access to it, and no single NS will be allowed to delete or change it without a consensus being reached. Minor changes to the standard LoRaWAN join procedure are necessary as join and re-join messages are encrypted in the proposed approach. This, of course, ensures extra confidentiality to the join procedure. Although the proof-of-concept implementation is based on Ethereum technology, the approach does not exploit the public Ethereum network but a private blockchain network, with the NSs operating as miners.

In [17], [18], authors contributed to a BC/SC-backed name resolution for a Serving NS (sNS) together with a new security model that adds non-repudiation to the LoRaWAN standard protocol using digital signatures. The SCs are implemented on top of the public Ethereum blockchain, providing a key/value store that maps JoinEUIs to NS addresses. Thus, the SCs also provide implicit roaming agreements. The solution, however,

requires that participants run their NSs in addition to the AS. The proposed security model employs two key pairs for EDs and NSs authentication and an additional symmetric key for data confidentiality. Of course, the proposed model is not compliant with the LoRaWAN standard protocol, which is based on symmetric cryptography, and would require extensive modifications to (e.g., legacy) firmware. Where available, the possibility of leveraging hardware security modules to accelerate cryptographic operations is particularly interesting.

Finally, in [19], [20], authors focused on key management issues in LoRaWAN, which, in the standard LoRaWAN architecture, become a single point of failure. The problem is tackled by designing and implementing an approach integrating the LoRaWAN Join Server (JS) with a private and permissioned blockchain to store the cryptographic keys of the EDs. This guarantees decentralization, high availability, security, integrity verification of transactions, immutability, and audibility.

III. Roaming Approaches

There are two types of roaming in LoRaWAN: passive and handover roaming. This section summarizes these two roaming modes of operation, as defined by the LoRa Alliance [21].

A. Passive Roaming

In passive roaming, an ED may belong to an NS even though packets originating from that ED actually travel across a GW under the control of a different NS. Thus, LoRa Session (and MAC-layer) control of the ED is performed by the first NS, denoted as sNS. Frame forwarding is done by the second NS, denoted as Forwarding NS (fNS). In a LoRa Session only one sNS is allowed, whereas the number of fNSs is not limited to one (and even no fNS is an option).

Regarding passive roaming, two types are defined: stateful and stateless.

1) *Stateful Roaming:* an fNS creates a context for an ED, used for processing (subsequent) packets going to, or coming from, the same ED.
2) *Stateless Roaming:* an fNS does not create any context and, consequently, each packet is processed independently.

However, the Lora Alliance does not describe the mechanism for which a roaming partner gets to know a given fNS. This leaves the door open for the availability of different mechanisms supporting roaming procedures but not cross-compatible. Passive roaming is supported by both LoRaWAN 1.0 and 1.1 specifications. Thus, implementations based on passive roaming should be technically feasible.

Figure 1 illustrates how passive roaming works for the first (incoming) packet from an ED: when a packet that does not belong to the Network Server 2 (NS2) provider reaches the latter, NS2 extracts the network ID from the incoming packet (NwkID), proceeds to look up the NetID (NwkID is the seven least significant bits of a registered 24-bit NetID) and checks the roaming policy. After identifying the sNS (NS1), NS2 sends a message to NS1 carrying the packet payload and other

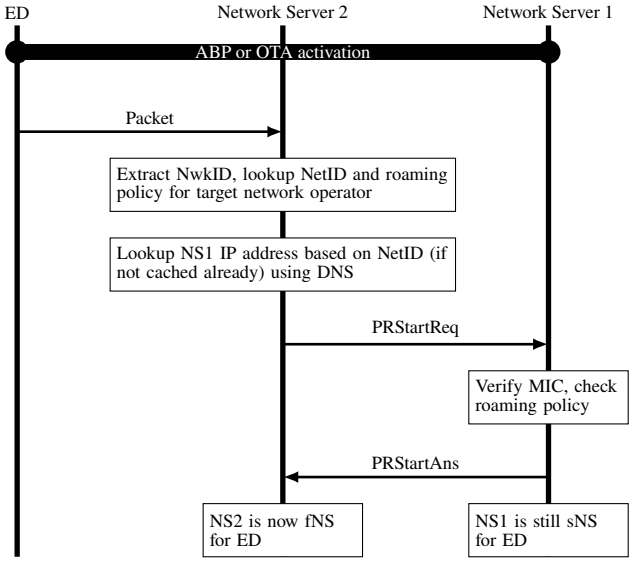


Fig. 1. Passive roaming: set up procedure.

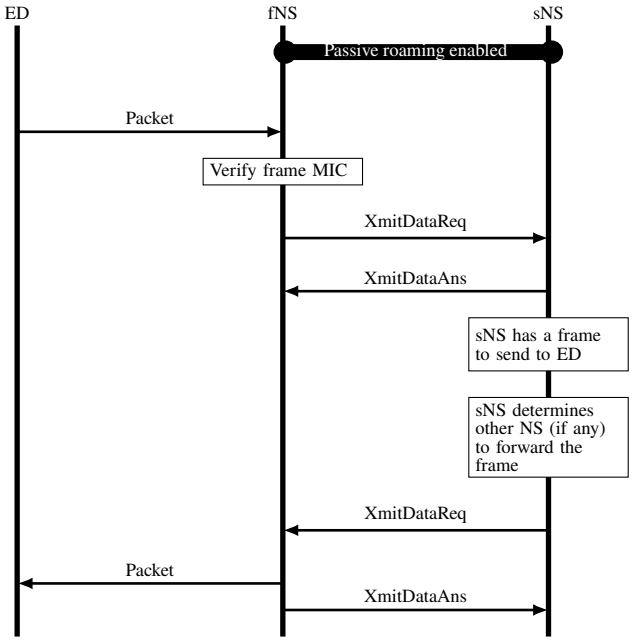


Fig. 2. Passive roaming: packet transmission procedure.

relevant fields. Then, NS1 verifies the packet Message Integrity Code (MIC) and checks the roaming policy as well. If no issue is found, NS1 replies with a PRStartAns message, thus designating NS2 as fNS of the ED.

Once passive roaming has been established, forwarding subsequent packets is more straightforward, as shown in Figure 2. In this case, the fNS only verifies MICs of incoming packets and sends an XmitDataReq carrying the packet payload, and other relevant fields, to the sNS. The sNS then checks whether there is a pending (incoming from other networks, e.g., from the right-hand side) frame to be transmitted to the ED, and if the frame just MIC-verified has to be forwarded to another NS

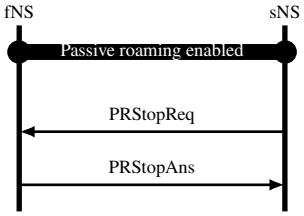


Fig. 3. Passive roaming: teardown procedure, initiated by the sNS.

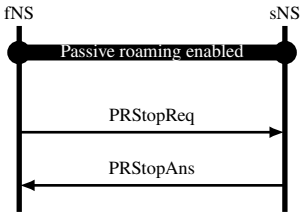


Fig. 4. Passive roaming: teardown procedure, initiated by the fNS.

belonging to the same provider as the sNS. If there is a pending packet to be transmitted, the sNS sends an XmitDataReq to the fNS, which forwards it to the ED.

Passive roaming may be torn down anytime. This procedure can be initiated by either the fNS or the sNS, as shown in Figures 3 and 4, respectively. In this case, the NS, in charge of tearing down roaming, sends a PRStopReq to the other NS, which answers with a PRStopAns.

B. Handover Roaming

Handover roaming enables the transfer of the MAC layer control from one NS to another [21]. Thus, even after an ED performs a handover roaming, the sNS keeps the control-plane and the data-plane together with the JS and the AS. Consequently, the sNS does not change for a given LoRa session until a new join procedure is performed by the ED. In this case, the sNS also controls that ED radio frequency (RF) settings. This type of roaming is only supported by LoRaWAN 1.1. Considering that LoRaWAN 1.1 is technically not viable and that the LoRa Alliance does not certify LoRaWAN 1.1 devices, handover roaming may be assumed as being unavailable (nor planned) at the time of drafting this paper.

C. LoRaWAN Roaming Is Not A Solution

LoRaWAN Roaming is primarily designed for first-generation public networks. At the same time, LoRaWAN traffic exchange is more powerful in a blend of private (business) networks, public collaborative and managed networks, and even gateways that a network service provider does not manage. LoRaWAN Roaming is not a good solution because it has five major shortcomings in its design:

- It requires blind trust in roaming parties or hubs;
- It requires many-to-many contracts;
- It does not support separating payload from metadata;
- It requires each party to operate a LoRaWAN NS;
- Roaming hubs centralize authority.

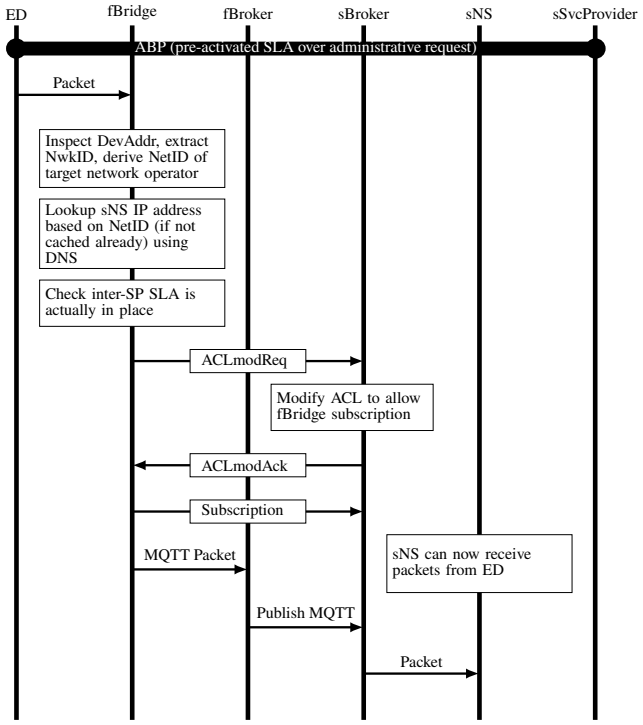


Fig. 5. Proposed ABP-based data packet roaming (without SLA setup).

IV. Architecture

This paper aims to provide a roaming solution for EDs moving across "GW Networks" (GWs belonging to different operators), designating *forwarding* GWs (available on-demand) that can "join" the sNS the roaming ED belongs to, in addition to their previously joined (home) NSs.

A. Roaming Interactions

The proposed solution, unlike passive roaming, does not involve any interaction among NSs and just works on top of services running on the GW, in charge of "mangling" incoming LoRa packets and forwarding them to the appropriate sNS. In Figure 5, a sequence of steps encompassed in the proposed roaming is depicted. As soon as LoRa packets arrive in fBridge, they are decoded to obtain the Device Address (DevAddr) of the ED who generated them, through which it is possible to derive the NwkID and NetID to which they belong to. To get the IP address of the sNS, if it is not cached already, a DNS request is generated using the value of the resolved NetID as a parameter in the request. Once sNS' IP address is available, the GW is ready to transmit if authorized in accordance with an active SLA with the sBroker. If it is not allowed, it is possible to send an Access Control List (ACL) modification request (ACLmodReq) to be enabled to use the service of sBroker. In Figure 6, the same request could be performed through automatic negotiation and establishment of inter-ServiceProvider SLA through an SC with the Service Provider. Once the ACLmodReq is answered (ACLmodAck), the fBridge can subscribe to the sBroker, and then the fBroker

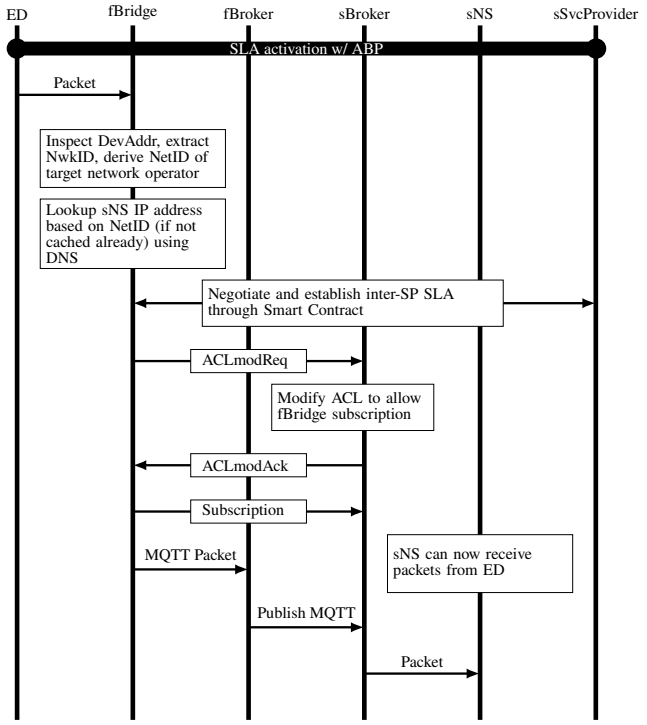


Fig. 6. Proposed ABP-based data packet roaming (including SLA setup).

is ready to forward the mangled MQTT packets, publishing them to the sNS. The proposed solution may enable several interesting use cases, such as:

- **Virtual LoRa operators:** Providers without a physical network (e.g., gateways/antennas) of their own, yet offering connectivity services to their customers.
- **Exchange of telemetry information among providers:** LoRa network operators may exchange telemetry information (e.g., no end-user data involved) passively, originating from roaming EDs. This information can be helpful in applications such as localization and coverage assessment.

B. Agreement Protocols

An SLA is a commitment between two actors about the conditions and the costs of a service. In the proposed architecture, actors are the serving Service Provider (sSvcProvider) and the forwarding Service Provider (fSvcProvider). The sSvcProvider is the entity that manages the sNS and the sBridge. On the other hand, the fSvcProvider, possibly located elsewhere, manages the fNS and the fBridge. Both actors are involved in the establishment of an SLA agreement, that may be negotiated through SCs.

During SLA setup, the fBridge is the main component of the architecture, in charge, as a LoRa frame arrives, of checking the presence and validity of an SLA. In its absence, it triggers the negotiation between the two SCs, Serving Smart Contract (sSC) and the Forwarding Smart Contract (fSC), based on policies, previously established by the Service Providers' governance, related to usable data traffic or goodput.

Examples of policies may be related to signalling traffic and how it is accounted, for instance, to the counterparties, so that the foreign end-user is charged only the cost referred to the payload of the requested/received packets. Or these could be about the lump sum quantity of packets that can be purchased and the related price thresholds, or similar policies. It is possible to reach an SLA agreement, only when the clauses on which it can have a match are composed of a non-null set. If the match is confirmed, then the fSvcProvider charges the sSvcProvider the cost agreed by the SC, and at the same time, to unlock packet forwarding it is necessary to instruct the fBridge to allow data traffic. This is done by letting the sSC issue an authorization Token for the fBridge, as the sBroker is equipped with an ACL. Since the information written on the SC is in clear text, the Token is encrypted using the public key of the fbridge, the same which is used (in optional mode) for MQTT communication.

To avoid malicious behaviour by any of the actors of the architecture, and therefore avoid potential threats, SCs will be also used as a logger. Each packet sent will be counted employing a counter in the fSC, and similarly, for each packet received, a counter residing in the sSC will be incremented. When credit for data traffic gets close to complete depletion, and thus the SLA Agreement as well, the fSvcProvider notifies the sSvcProvider that credit is running out. Subsequently, a check is carried out on (potential) inconsistencies out of the packets count by the logger within the SCs, and if the gap (which most likely would be accounted to packet loss) is greater than a pre-established threshold about the Quality of Service (QoS), it is possible to act in different ways, for instance by requesting a refund as an SLA violation has been reported, because of the terms of the SC have not been respected, or triggering a downtrending renegotiation of the SC for subsequent agreements.

V. Proof-of-concept Implementation and Experimental Results

A. Implementation

The standard LoRaWAN passive roaming protocol is here adapted to enable a roaming connection without having preset agreements among NSs. The idea is to allow an ED to interact with a forwarding GW and, through it, to reach its sNS. To achieve this, the GW has been modified to resolve the NwkID and NetID of the LoRa packet received from the ED. The resolution of the DevAddr can be conducted by extracting the PhyPayload key value from the received frame and decoding it. Later, the NetID can be obtained from the extracted DevAddr, computing it with the aid of a Look-Up Table containing details about the LoRa Alliance NetID assignments. A DNS request is executed to obtain the IP address of the related NS, using the NetID as a parameter. This process works akin to the LoRa DNS proposed in [3].

Moreover, a Mosquitto Broker is provided inside the GW, in charge of forwarding MQTT packets and publishing the MQTT message to the corresponding sNS' Broker.

The MQTT messages, forwarded by the fBroker, are first mangled to let the packets be accepted by the sNS. The packet mangling consists of replacing (inside them) the GatewayID (unique identification for a specific Gateway) of the forwarding GW with a previously known GatewayID corresponding to the specific GW for the specific ED. The needed GatewayID, if not cached already, may be obtained during the SLA agreement. The whole workflow is illustrated in Figure 7.

The proof-of-concept for the proposed solution has been set up using an open-source, community-driven, reference LoRaWAN implementation, ChirpStack [22], and an ED/GW simulator [23].

We opted to utilize an ED simulator since the proposed solution does not impact the radio frequency transmission. Consequently, the emulation is considered sufficient, and it is not necessary to replicate the whole architecture on a real testbed.

B. Experimental Results

Experiments have been conducted with the support of a simulator, which can create virtual EDs and GWs, pointing to a physical GW Bridge. The testbed comprises two physical machines equipped with 8-core processors and 32GB RAM, and a Raspberry PI4. One machine runs the tool that consents to simulate the desired number of EDs. The other machine runs the Cloud component of ChirpStack, specifically the NS and the sBroker. Moreover, the Raspberry Pi4 performs as GW and hosts the GW-Bridge service (proposed in this document). We performed two tests, varying in how the GW on the Raspberry is configured. In the first setup (standard solution), the GW-Bridge has the sBroker as the MQTT server end-point. In the second setup, the end-point is set on the local fBroker instead.

To evaluate our LoRa roaming proposal, we first computed the LoRa packets' travel time elapsed, performing the standard LoRa Passive roaming solution in our testbed to have parameters for comparison. The passive roaming configuration has been set up properly, as the first setup, before the experiment so that to have the simulated EDs that send data to the sNS via the forwarding GW. The passive roaming outcomes are listed in Table II. In the second setup, which involves our roaming proposal, a forwarding broker (fBroker) running on the Raspberry has been initialized, and an ad-hoc service has been deployed, responsible for decoding LoRaWAN packets. As the first packet arrives, the DevAddr is extracted, and the NetID is resolved to obtain the IP address of the sNS via DNS. We used Node.js "Lora-packet" library to perform tests in decoding LoRa packets, and our experimental result demonstrated that the average data extraction time has a cost of about 430 ms. Then to improve the performance, we implemented our solution in such a way as to operate packet decoding only once for each ED's transmission session, caching needed parameters in the local memory.

The second step focuses on manipulating MQTT Packets to let sNS identify the packets forwarded from an unidentified GW as if they were forwarded from an already registered GW.

TABLE II
CHIRPSTACK PASSIVE ROAMING.

Packets travel time		
Number of EDs	ED to sNS	Confidence intervals up to 95%
10	0 ms	0 ms
20	0 ms	0 ms
50	0 ms	0 ms
100	0 ms	0 ms
250	311.15 ms	± 3.97 ms
500	348.84 ms	± 4.11 ms
1000	402.60 ms	± 3.47 ms

TABLE III
PROPOSED ROAMING SOLUTION.

Packets travel and handling time				
Number of EDs	ED to sNS	Confidence intervals up to 95%	Handling packets	Confidence intervals up to 95%
10	0 ms	0 ms	0 ms	0 ms
20	0 ms	0 ms	0 ms	0 ms
50	0 ms	0 ms	0 ms	0 ms
100	313.05 ms	± 15.3 ms	90.33 ms	± 7.33 ms
250	413.57 ms	± 16.2 ms	82.78 ms	± 6.11 ms
500	676.64 ms	± 20.95 ms	76.02 ms	± 5.48 ms
1000	1822.1 ms	± 28.54 ms	72.48 ms	± 3.18 ms

Finally, the frames are published to the sBroker and are available for logging into the AS. Results in terms of packet travel time are reported in Table III.

Some important considerations can be made based on the results obtained from the simulations. In Table II, we can evaluate the standard LoRa passive roaming solution. It has a linear trend with an average packet travel time for less than 250 devices close to 300 ms. When this threshold is exceeded, the packet travel time increases by approximately 100 ms. Compared to the standard passive roaming solution, the solution we propose automates the forwarding of LoRa packets to the sNS by the GW, thus requiring a more significant computational load to the GW. Outcomes in Table III confirm the expected results. The simulations demonstrate that for less than 250 devices, the computational load is manageable by an embedded device such as the Raspberry PI 4 used in the testbed. The results obtained differ from the passive roaming performance of about 100 ms, which corresponds, on average, to the time taken by the GW to handle the packets. On the other hand, the results for a number of devices greater than one hundred progressively worsen due to the computational load introduced. However, since LoRa architectures are typically employed in solutions where devices have a low packet send rate (even minutes), this solution introduces an acceptable delay in any case. The proposed solution (which allows roaming operating at the GW level) has a slightly higher transmission time concerning the standard solution.

VI. CONCLUSION

Roaming over LoRaWAN may catalyze the emergence of added-value applications and services based on this technology. The current LoRa specifications already contemplate implementing roaming, although with remarkable gaps and

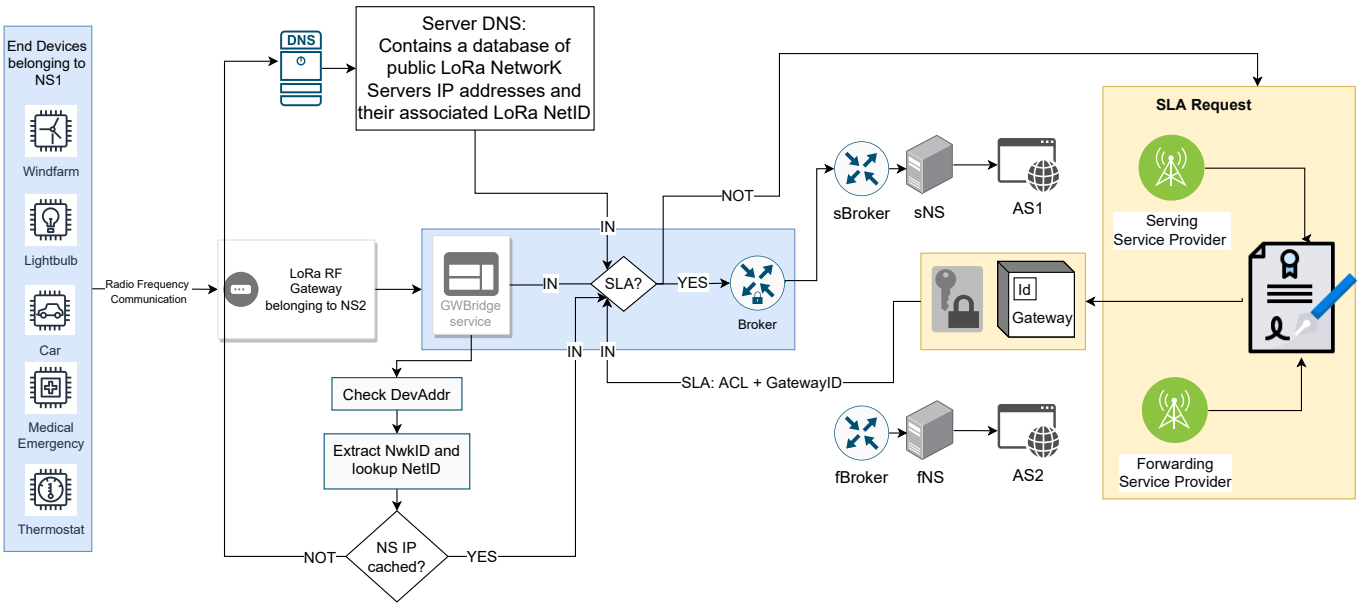


Fig. 7. Workflow for the proposed solution ABP-based data packet roaming (including SLA setup).

shortcomings. Indeed, roaming mechanisms in the 1.0 specification are very limited, whereas no device certification program is currently available for the 1.1 version of the spec. Furthermore, due to the huge market penetration of v1.0-compliant devices, it seems very unlikely that a transition to v1.1 of the spec will happen anytime soon.

In any case, the limitations of the 1.0 specification should not be a hindrance to the adoption of feature-rich, efficient roaming schemes based on inter-provider agreements. The architecture proposed in this article is designed to: i) decouple the deployment and operations of GWs from those of (LoRaWAN) networks; ii) establish Gateway-to-Network connections on-demand, according to actual (incoming) traffic due to mobility; iii) support agreements dynamically, by automatically settling provisions and fees through SCs.

Moreover, the preliminary tests presented in this work, and carried out through simulations, validate the architecture's suitability and gauge packet handling times while roaming. The approach presented in this paper involves employing an additional broker at the Gateway level. Yet, the approach overall fits with the Cloud Continuum paradigm since alternative configurations can be devised, where the additional broker can be deployed at different levels of the LoRa stack.

We currently have an ongoing collaboration with a telco providers' national association, AssoProvider, and we plan to have the solution described in this document deployed in trials nationwide.

Acknowledgment

This research was supported by project AriSe2, ref. grant PID2020-116329GB-C22, and project SmartDUMP, ref. grant TED2021-129336B-I00, funded by MCIN/AEI/10.13039/501100011033 and European Union NextGenerationEU/PRTR.

References

[1] I. Butun, N. Pereira, and M. Gidlund, "Analysis of lorawan v1.1 security: Research paper," in *Proceedings of the 4th ACM MobiHoc Workshop on Experiences with the Design and Implementation of Smart Objects*, ser. SMARTOBJECTS '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: https://doi.org/10.1145/3213299.3213304

[2] ——, "Security risk analysis of lorawan and future directions," *Future Internet*, vol. 11, no. 1, 2019. [Online]. Available: https://www.mdpi.com/1999-5903/11/1/3

[3] S. Balakrichenan, A. Bernard, M. Marot, and B. Ampeau, "Iotroam - design and implementation of an open lorawan roaming architecture," 2021, pp. 1–7.

[4] A. Lemogue, I. Martinez, L. Toutain, and A. Bouabdallah, "Federated iot roaming using private dns resolutions," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, 2022, pp. 1–6.

[5] E. M. Torroglosa-Garcia, J. M. A. Calero, J. B. Bernabe, and A. Skarmeta, "Enabling roaming across heterogeneous iot wireless networks: Lorawan meets 5g," *IEEE Access*, vol. 8, pp. 103 164–103 180, 2020.

[6] W. Ayoub, F. Nouvel, A. E. Samhat, M. Mroue, and J.-C. Prévotet, "Mobility management with session continuity during handover in lpwan," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 6686–6703, 2020.

[7] S. Bala, D. Barthel, and S. Gharout, "Separate session key generation approach for network and application flows in lorawan," in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 870–879. [Online]. Available: https://doi.org/10.1145/3297280.3297366

[8] F. Flammini, A. Gaglione, D. Tokody, and D. Dohrilovic, "Lora wan roaming for intelligent shipment tracking," 2020, pp. 1–2.

[9] M. Hamnache, R. Kacimi, and A.-L. Beylot, "Unifying lorawan networks by enabling the roaming capability," 2021, pp. 371–374.

[10] A. Bernard, M. Laroui, M. Marot, S. Balakrichenan, H. Moungla, B. Ampeau, H. Afifi, and M. Becker, "Prefetching of mobile devices information - a dns perspective," in *ICC 2022 - IEEE International Conference on Communications*, 2022, pp. 4293–4299.

[11] J. Lin, Z. Shen, C. Miao, and S. Liu, "Using blockchain to build trusted lorawan sharing server," *International Journal of Crowd Science*, 2017.

[12] J. Lin, Z. Shen, and C. Miao, "Using blockchain technology to build trust in sharing lorawan iot," in *Proceedings of the 2nd International Conference on Crowd Science and Engineering*, 2017, pp. 38–43.

[13] N. S. Mtetwa, N. Sibeko, P. Tarwireyi, and A. M. Abu-Mahfouz, "OTA firmware updates for LoRaWAN using blockchain," in *2020 2nd International Multidisciplinary Information Technology and Engineering Conference (IMITEC)*. IEEE, 2020, pp. 1–8.

[14] H. Jradi, A. E. Samhat, F. Nouvel, M. Mroue, and J.-C. Prévotet, "Overview of the mobility related security challenges in lpwans," *Computer Networks*, p. 107761, 2020.

[15] S. M. Danish, M. Lestas, W. Asif, H. K. Qureshi, and M. Rajarajan, "A lightweight blockchain based two factor authentication mechanism for LoRaWAN join procedure," in *2019 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2019, pp. 1–6.

[16] S. M. Danish, M. Lestas, H. K. Qureshi, K. Zhang, W. Asif, and M. Rajarajan, "Securing the LoRaWAN join procedure using blockchains," *Cluster Computing*, vol. 23, no. 3, pp. 2123–2138, 2020.

[17] A. Durand, P. Gremaud, and J. Pasquier, "Resilient, crowd-sourced LPWAN infrastructure using blockchain," in *Proceedings of the 1st Workshop on Cryptocurrencies and Blockchains for Distributed Systems*, 2018, pp. 25–29.

[18] ——, "Decentralized LPWAN infrastructure using blockchain and digital signatures," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 12, p. e5352, 2020.

[19] V. Ribeiro, R. Holanda Filho, and A. Ramos, "A Secure and Fault-Tolerant Architecture for LoRaWAN Based on Blockchain," in *2019 3rd Cyber Security in Networking Conference (CSNet)*. IEEE, 2019, pp. 35–41.

[20] V. Ribeiro, R. Holanda, A. Ramos, and J. J. Rodrigues, "Enhancing Key Management in LoRaWAN with Permissioned Blockchain," *Sensors*, vol. 20, no. 11, p. 3068, 2020.

[21] "TS2-1.1.0 LoRaWAN® Backend Interfaces," Jan 2021. [Online]. Available: https://lora-alliance.org/resource_hub/ts002-110-lorawan-backend-interfaces/

[22] "Chirpstack, open-source lorawan® network server stack." [Online]. Available: https://www.chirpstack.io/

[23] UniCT-ARSLab, "Unict-arslab/lwn-simulator: A lorawan nodes' and network simulator." [Online]. Available: https://github.com/UniCT-ARSLab/LWN-Simulator